

of the uncentered estimator. By Theorem 1 (the uncentered estimator is unbiased), the centered estimator is therefore also unbiased, which completes the proof of the first statement.

Regarding the optimal choice of $\bar{L}_i$, first note that the variance of the uncentered estimator is

$$\text{Var}[(h_i - \text{sigm}(a_i))L] = E[(h_i - \text{sigm}(a_i))^2 L^2] - E[\hat{g}_i]^2.$$

Now let us compute the variance of the centered estimator:

$$\begin{aligned} \text{Var}[(h_i - \text{sigm}(a_i))(L - \bar{L}_i)] &= E[(h_i - \text{sigm}(a_i))^2 (L - \bar{L}_i)^2] - E[(h_i - \text{sigm}(a_i))(L - \bar{L}_i)]^2 \\ &= E[(h_i - \text{sigm}(a_i))^2 L^2] + E[(h_i - \text{sigm}(a_i))^2 \bar{L}_i^2] \\ &\quad - 2E[(h_i - \text{sigm}(a_i))^2 L \bar{L}_i] - (E[\hat{g}_i] - 0)^2 \\ &= \text{Var}[(h_i - \text{sigm}(a_i))L] - \Delta \end{aligned} \tag{11}$$

where $\Delta = 2E[(h_i - \text{sigm}(a_i))^2 L \bar{L}_i] - E[(h_i - \text{sigm}(a_i))^2 \bar{L}_i^2]$. Let us rewrite Δ :

$$\begin{aligned} \Delta &= 2E[(h_i - \text{sigm}(a_i))^2 L \bar{L}_i] - E[(h_i - \text{sigm}(a_i))^2 \bar{L}_i^2] \\ &= E[(h_i - \text{sigm}(a_i))^2 \bar{L}_i (2L - \bar{L}_i)] \\ &= E[(h_i - \text{sigm}(a_i))^2 (L^2 - (L - \bar{L}_i)^2)] \end{aligned} \tag{12}$$

Δ is maximized (to minimize variance of the estimator) when $E[(h_i - \text{sigm}(a_i))^2 (L - \bar{L}_i)^2]$ is minimized. Taking the derivative of that expression with respect to $\bar{L}_i$, we obtain

$$2E[(h_i - \text{sigm}(a_i))^2 (\bar{L}_i - L)] = 0$$

which, as claimed, is achieved for

$$\bar{L}_i = \frac{E[(h_i - \text{sigm}(a_i))^2 L]}{E[(h_i - \text{sigm}(a_i))^2]}.$$

□

Note that a general formula for the lowest variance estimator for REINFORCE (Williams, 1992) had already been introduced in the reinforcement learning context by Weaver and Tao (2001), which includes the above result as a special case. This followed from previous work (Dayan, 1990) for the case of binary immediate reward.

Practically, we could get the lowest variance estimator (among all choices of the $\bar{L}_i$) by keeping track of two numbers (running or moving averages) for each stochastic neuron, one for the numerator and one for the denominator of the unit-specific $\bar{L}_i$ in Eq. 10. This would lead the lowest-variance estimator $(h_i - \text{sigm}(a_i))(L - \bar{L}_i)$. Note how the unbiased estimator only requires broadcasting L throughout the network, no back-propagation and only local computation. Note also how this could be applied even with an estimate of future rewards or losses L , as would be useful in the context of reinforcement learning (where the actual loss or reward will be measured farther into the future, much after h_i has been sampled).

4 Straight-Through Estimator

Another estimator of the expected gradient through stochastic neurons was proposed by Hinton (2012) in his lecture 15b. The idea is simply to back-propagate through the hard threshold function (1 if the argument is positive, 0 otherwise) as if it had been the identity function. It is clearly a biased estimator, but when considering a single layer of neurons, it has the right sign (this is not guaranteed anymore when back-propagating through more hidden layers). We call it the **straight-through** (ST) estimator. A possible variant investigated here multiplies the gradient on h_i by the derivative of the sigmoid. Better results were actually obtained without multiplying by the derivative of the sigmoid. With h_i sampled as per Eq. 3, the straight-through estimator of the gradient of the loss L with respect to the pre-sigmoid activation a_i is thus

$$g_i = \frac{\partial L}{\partial h_i}. \tag{13}$$

Like the other estimators, it is then back-propagated to obtain gradients on the parameters that influence a_i .

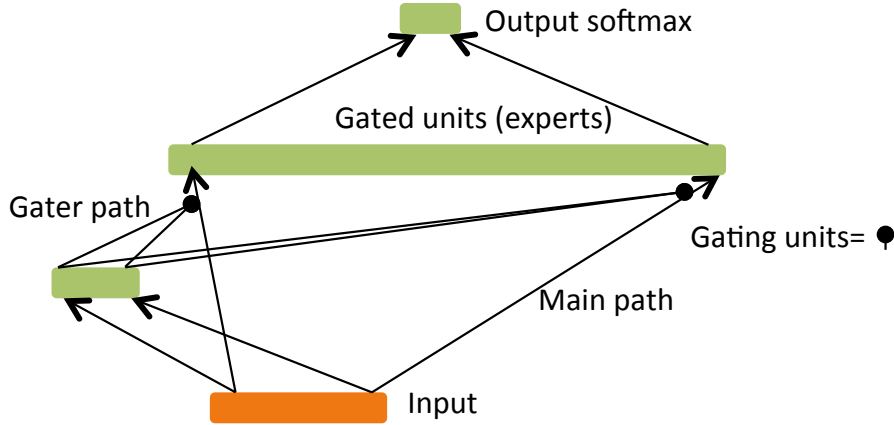
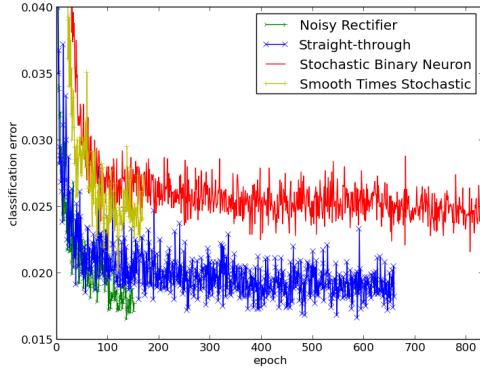


Figure 1: Conditional computation architecture: gater path on the left produces sparse outputs h_i which are multiplied elementwise by the expert outputs, hidden units H_i .

5 Conditional Computation Experiments

We consider the potential application of stochastic neurons in the context of conditional computation, where the stochastic neurons are used to select which parts of some computational graph should be actually computed, given the current input. The particular architecture we experimented with is a neural network with a large hidden layer whose units H_i will be selectively turned off by gating units h_i , i.e., the output of the i -th hidden unit is $H_i h_i$, as illustrated in Figure 1. For this to make sense in the context of conditional computation, we want h_i to be non-zero only a small fraction α of the time (10% in the experiments) while the amount of computation required to compute h_i should be much less than the amount of computation required to compute H_i . In this way, we can first compute h_i and only compute H_i if $h_i \neq 0$, thereby saving much computation. To achieve this, we connect the previous layer to h_i through a bottleneck layer. Hence if the previous layer (or the input) has size N and the main path layer also has size N (i.e., $i \in \{1, \dots, N\}$), and the gater’s bottleneck layer has size $M \ll N$, then computing the gater output is $O(MN)$, which can reduce the main path computations from $O(N^2)$ to $O(\alpha N^2)$.



	train	valid	test
Noisy Rectifier	6.7e-4	1.52	1.87
Straight-through	3.3e-3	1.42	1.39
Smooth Times Stoch.	4.4e-3	1.86	1.96
Stoch. Binary Neuron	9.9e-3	1.78	1.89
Baseline Rectifier	6.3e-5	1.66	1.60
Baseline Sigmoid+Noise	1.8e-3	1.88	1.87
Baseline Sigmoid	3.2e-3	1.97	1.92

Figure 2: Left: Learning curves for the various stochastic approaches applied to the MNIST dataset. The y-axis indicates classification error on the validation set. Right: Comparison of training criteria and valid/test classification error (%) for stochastic (top) and baseline (bottom) gaters.

Experiments are performed with a gater of 400 hidden units and 2000 output units. The main path also has 2000 hidden units. A sparsity constraint is imposed on the 2000 gater output units such that each is non-zero 10% of the time, on average. The validation and test errors of the stochastic models are obtained after optimizing a threshold in their deterministic counterpart used during testing. See the appendix in supplementary material for more details on the experiments.

For this experiment, we compare with 3 baselines. The Baseline Rectifier is just like the noisy rectifier, but with 0 noise, and is also constrained to have a sparsity of 10%. The Baseline Sigmoid is like the STS and ST models in that the gater uses sigmoids for its output and tanh for its hidden

units, but has no sparsity constraint and only 200 output units for the gater and main part. The baseline has the same hypothetical resource constraint in terms of computational efficiency (runtime), but has less total parameters (memory). The Baseline Sigmoid with Noise is the same, but with Gaussian noise added during training.

6 Conclusion

In this paper, we have motivated estimators of the gradient through highly non-linear non-differentiable functions (such as those corresponding to an indicator function), especially in networks involving noise sources, such as neural networks with stochastic neurons. They can be useful as biologically motivated models and they might be useful for engineering (computational efficiency) reasons when trying to reduce computation via conditional computation or to reduce interactions between parameters via sparse updates (Bengio, 2013). We have proven interesting properties of three classes of stochastic neurons, the noisy rectifier, the STS unit, and the binary stochastic neuron, in particular showing the existence of an unbiased estimator of the gradient for the latter. Unlike the SPSSA (Spall, 1992) estimator, our estimator is unbiased even though the perturbations are not small (0 or 1), and it multiplies by the perturbation rather than dividing by it.

Experiments show that all the tested methods actually allow training to proceed. It is interesting to note that the gater with noisy rectifiers yielded better results than the one with the non-noisy baseline rectifiers. Similarly, the sigmoid baseline with noise performed better than without, *even on the training objective*: all these results suggest that injecting noise can be useful not just as a regularizer but also to help explore good parameters and fit the training objective. These results are particularly surprising for the stochastic binary neuron, which does not use any backprop for getting a signal into the gater, and opens the door to applications where no backprop signal is available. In terms of conditional computation, we see that the expected saving is achieved, without an important loss in performance. Another surprise is the good performance of the straight-through units, which provided the best validation and test error, and are very simple to implement.

Acknowledgments

The authors would like to acknowledge the useful comments from Guillaume Alain and funding from NSERC, Ubisoft, CIFAR (YB is a CIFAR Fellow), and the Canada Research Chairs.